

UNIT 1 — OOPs in C++

✅ 1. Object Oriented Programming (OOP)

Definition:

OOP is a programming method where program is designed using **objects and classes**.

👉 It represents real world concepts.

✅ 2. Object

Definition:

Object is an **instance of a class** which contains data and functions.

Example:

Student, Car, Mobile

```
class Student{};
Student s1;
```

✅ 3. Class

Definition:

Class is a **blueprint or template** used to create objects.

It contains:

- Data Members
- Member Functions

```
class Student{
public:
int roll;
void show();
};
```

✅ 4. Data Encapsulation

Definition:

Wrapping data and functions into a single unit is called encapsulation.

👉 Done using class.

✅ 5. Data Abstraction

Definition:

Showing only important information and hiding internal details.

Example: ATM Machine

✅ 6. Delegation

Definition:

When one object sends task to another object to complete work.

(Simple meaning → object helping another object)

✅ 7. Inheritance

Definition:

Creating a new class from existing class is called inheritance.

✅ 8. Polymorphism

Definition:

Same function name performing different tasks.

✅ 9. Message Communication

Definition:

Communication between objects using member functions.

Example:

`obj.show();`

✅ 10. Access Specifiers

Type	Meaning
Private	Access inside class only
Public	Access anywhere
Protected	Access in derived class

✓ 11. Constructor

Definition:

Constructor is special function automatically called when object is created.

Characteristics

- Same name as class
 - No return type
-

Types of Constructor

✓ Default Constructor

```
A(){}
```

✓ Parameterized Constructor

```
A(int x){}
```

✓ Copy Constructor

```
A(A &obj){}
```

✓ 12. Destructor

Definition:

Destructor destroys object and releases memory.

```
~A(){}
```

✓ 13. Friend Function

Definition:

Function that can access private data of class.

✓ 14. Friend Class

Definition:

A class allowed to access private members of another class.

UNIT 2 — Inheritance

✓ 1. Inheritance

Definition:

Process of acquiring properties of one class into another class.

✓ 2. Types of Inheritance

1. Single Inheritance
 2. Multiple Inheritance
 3. Multilevel Inheritance
 4. Hierarchical Inheritance
 5. Hybrid Inheritance
-

✓ 3. Visibility Modes

Mode	Result
------	--------

Public	Members remain public
--------	-----------------------

Protected	Become protected
-----------	------------------

Private	Become private
---------	----------------

✓ 4. Virtual Base Class

Definition:

Used to avoid duplication of data in multiple inheritance.

✓ 5. Benefits of Inheritance

- Code Reusability
 - Easy Maintenance
 - Saves Time
 - Program Extension
-

✓ 6. Virtual Function**Definition:**

Function declared using virtual keyword and executed at runtime.

```
virtual void show();
```

✓ 7. Pure Virtual Function**Definition:**

Virtual function with no definition.

```
virtual void show()=0;
```

✓ 8. Abstract Class**Definition:**

Class containing at least one pure virtual function.

Object cannot be created.

UNIT 3 — Polymorphism & Templates

✓ 1. Polymorphism

Definition:

Ability of function or operator to behave differently.

✓ 2. Types of Polymorphism**✓ Compile Time**

- Function Overloading
- Operator Overloading

✓ Run Time

- Virtual Function
-

✓ 3. Function Overloading**Definition:**

Using same function name with different parameters.

```
int add(int a,int b);
```

```
float add(float a,float b);
```

✓ 4. Operator Overloading**Definition:**

Giving special meaning to operators.

Unary Operator

Works on one operand.

Example:

```
++
```

Binary Operator

Works on two operands.

Example:

```
+
```

Using Friend Function

Operator defined outside class.

Without Friend Function

Operator defined inside class.

✓ 5. Function Template

Definition:

Generic function working with different data types.

```
template<class T>
T add(T a,T b);
```

✓ 6. Class Template

Definition:

Generic class used for multiple data types.

UNIT 4 — Exception Handling & Stream

✓ 1. Exception Handling

Definition:

Process of handling runtime errors.

✓ 2. Types of Exception

- Divide by Zero

- File Not Found
 - Memory Error
-

✓ 3. Exception Handling Keywords

Keyword Use

try Check error

throw Generate error

catch Handle error

```
try{  
    throw 10;  
}  
catch(int x){  
}
```

✓ 4. Stream Computation (Console)

Object Work

cin Input

cout Output

endl New line

✓ 5. File Stream

Header File:

```
#include<fstream>
```

Classes:

- ifstream → Read file
 - ofstream → Write file
 - fstream → Read & Write
-

✓ 6. File Read & Write

Write

```
ofstream file("data.txt");  
file<<"Hello";
```

Read

```
ifstream file("data.txt");  
file>>data;
```
